Network Security Analysis Report

Task 2: Network Security Analysis (SWYNEX Internship)

Prepared by: Fathima Hannah | Date: 5th Nov 2026

1. Introduction

The objective of this task was to analyze network traffic in a controlled environment using Wireshark. I captured traffic from my own Kali Linux lab, identified the protocols present, looked for suspicious patterns, and documented potential security concerns with defensive recommendations. No traffic was captured from networks I don't own or have permission to monitor.

2. Environment and Method

Item	Details
Tool	Wireshark (4.6.0.)
Operating system	Kali Linux (virtual machine)
Network mode	NAT
Interface captured	eth0
Kali machine IP	192.168.217.130
DNS server	192.168.217.235
Capture duration	about 3 minutes
Total packets captured	1470
Capture file	lab_capture.pcapng

Traffic generated during the capture:

- ICMP: `ping -c 4 8.8.8.8`
- DNS: `nslookup example.com`
- HTTP and HTTPS requests: `curl http://example.com` and `curl https://example.com`
- Browsing in Firefox: `http://neverssl.com` and `https://wikipedia.org`

Analysis steps:

1. Reviewed Statistics, then Protocol Hierarchy, to list the protocols present.
2. Applied display filters (`icmp`, `dns`, `http`, `tls`, `tcp`) to examine each protocol.
3. Used Follow, then HTTP Stream, to inspect unencrypted web traffic.
4. Reviewed Analyze, then Expert Information, for errors and warnings.

3. Protocols Observed

Protocol	Purpose	Filter used	Example packet No.	What it shows
ICMP	Tests connectivity (ping)	`icmp`	7 (request), 8 (reply)	Ping from Kali (192.168.217.130) to 8.8.8.8. The reply came back in packet 8. Four request/reply pairs were sent (packets 7 to 14).
DNS	Translates domain names to IP addresses	`dns`	17 (query), 18 (response)	Kali asked the DNS server (192.168.217.235) for example.com. The reply gave 172.66.147.243 and 104.20.23.154.
TCP	Reliable connections between devices	`tcp`	47, 48, 49	SYN, SYN-ACK, ACK handshake from Kali (port 37682) to 172.66.147.243 on port 443.
HTTP	Unencrypted web traffic	`http`	32 (request), 35 (reply)	GET / to 104.20.23.154 (example.com), answered with 200 OK (text/html), readable in plain text.
TLS	Encrypted web traffic (HTTPS)	`tls`	50 (Client Hello), 53 (Server Hello)	Secure connection to example.com (SNI shown in packet 50), followed by encrypted Application Data in packets 55 and 58.

4. Security Findings

Finding 1: Unencrypted HTTP traffic

- **Evidence:** Packet 32 is a plain-text HTTP GET / request to 104.20.23.154 (example.com), answered with 200 OK in packet 35. Using Follow, then HTTP Stream, the request and page content were readable. (Screenshot: `http_stream.png`)
- **Concern:** HTTP does not encrypt data. Anyone on the same network could read the content, and any credentials entered on an HTTP page would be exposed.
- **Risk level:** Medium

Finding 2: Encrypted HTTPS traffic (secure behavior)

- **Evidence:** Packet 50 is a TLS 1.3 Client Hello to example.com (server name visible), and packet 53 is the Server Hello. Packet 55 is Application Data with no readable content. (Screenshot: `tls_traffic.png`)
- **Observation:** Only the server name and connection details were visible. The page content stayed protected. This is the expected secure behavior.
- **Risk level:** Low

Finding 3: DNS queries are visible

- **Evidence:** Packet 17 is a DNS query for example.com, and packet 18 is the server's reply with the IP addresses. (Screenshot: `dns_queries.png`)
- **Concern:** Standard DNS is unencrypted, so an observer on the network can see which sites a device visits, even when the sites use HTTPS.
- **Risk level:** Low

Finding 4: TCP duplicate ACK and connection teardown

- **Evidence:** Packet 40 is flagged as a TCP Dup ACK, followed by the normal connection close in packets 41 (FIN, ACK) and 42 (ACK). (Screenshot: `expert_info.png`)
- **Observation:** This is the normal way a web connection closes. It is not an attack.
- **Risk level:** Low

Other checks performed

I also checked for common suspicious patterns and found none in this capture:

- Port scanning (many SYN packets from one host to many ports)
- ARP spoofing (duplicate IP-to-MAC mappings)
- Unusual or very long DNS names (possible tunneling)
- Regular repeated connections to one external IP (possible beaconing)
- Large unexpected outbound transfers

No scan, spoofing, or beaconing pattern appeared in the filtered views I reviewed (ICMP, DNS, HTTP, TLS).

5. Risk Summary

#	Finding	Packets	Risk level
1	Unencrypted HTTP traffic	32, 35	Medium
2	Encrypted HTTPS traffic (secure)	47 to 59	Low
3	Visible DNS queries	17, 18	Low
4	TCP duplicate ACK and teardown	40 to 42	Low

6. Recommendations

1. Use HTTPS everywhere. Enable HTTPS-only mode in browsers and avoid sending data over HTTP.
2. Never enter passwords on HTTP pages. Credentials sent over HTTP can be read by anyone on the network.
3. Use encrypted DNS (DNS over HTTPS or DNS over TLS) to hide domain lookups.
4. Use a firewall to block unnecessary ports and restrict outbound traffic.
5. Monitor network traffic with an IDS such as Snort or Suricata to detect scans and unusual patterns.

6. Segment the network so a compromised device can't reach everything.
7. Use a VPN on untrusted networks.

7. Conclusion

The captured traffic consisted of ICMP, DNS, TCP, HTTP, and TLS. The main security concern was unencrypted HTTP traffic, which exposes content to anyone on the network, while HTTPS traffic was properly encrypted. No malicious activity was found in this lab capture. This task showed how Wireshark can be used defensively to identify protocols, spot weak security practices, and recommend improvements.

Appendix: Evidence Files

- `lab_capture.pcapng`: the packet capture
- `screenshots/protocol_hierarchy.png`
- `screenshots/icmp.png`, `dns_queries.png`, `http_stream.png`, `tls_traffic.png`, `tcp.png`
- `screenshots/expert_info.png`